A04
CRYPTOGRAPHIC FAILURES
Broken transport, weak hashing,
bad cipher choices, exposed keys

A08
SOFTWARE & DATA INTEGRITY FAILURES
Tampered inputs, forged tokens,
unsigned updates, unsafe file uploads

A04 Cryptographic Failures — Four Failure Contexts & How to Fix Them

Context	Common Mistake	Why It Fails	Correct Pattern

TRANSPORT (TLS)	TLS 1.0/1.1 allowed; self-signed cert accepted; no HSTS	Deprecated (RFC 8996). BEAST/POODLE attacks. First HTTP req exposes cookie.	`ssl_protocols TLSv1.2 TLSv1.3;`
			`ssl_ciphers ECDHE-...-GCM-SHA384;`
			Flask-Talisman: HSTS max-age=31536000. Verify: SSLLabs.com A+.

PASSWORD STORAGE	MD5, SHA-1 or unsalted SHA-256; rounds too low	MD5: ~10 billion hashes/sec on GPU. All of rockyou.txt cracks in < 1 sec.	`bcrypt.hashpw(pwd.encode(), bcrypt.gensalt(rounds=12))` Alternatives: Argon2 (argon2-cffi), scrypt. Never rounds < 10.

APP DATA ENCRYPTION	AES-ECB mode; hardcoded IV; hardcoded key in source code	ECB: identical plaintext blocks → identical ciphertext (the 'ECB penguin'). Hardcoded key = permanent breach.	`from cryptography.fernet import Fernet`
			`key = os.environ['ENCRYPTION_KEY']`
			`f = Fernet(key); f.encrypt(b'data')`
			Fernet = AES-128-CBC + HMAC-SHA256. Never implement your own cipher.

TOKENS / JWTs / KEYS	Weak or hardcoded JWT secret; alg:none accepted; secret committed to git	alg:none removes signature entirely. Git history retains secrets even after deletion — GitHub's secret scanner may have already indexed it.	`SECRET = os.environ['JWT_SECRET'] # 32+ bytes`
			`token = jwt.encode(payload, SECRET, algorithm='HS256')`
			`jwt.decode(token, SECRET, algorithms=['HS256'])`
			Always pass algorithms=[...]. Never pass options={'verify_signature': False}. Rotate immediately on exposure.

Certificate Transparency (CT) — How the Trust Model Works

What is CT?	How to use crt.sh	Defensive Use

Every public TLS certificate must be submitted to at least two CT Logs before browsers will trust it. Logs are public, append-only, and cryptographically auditable.
A mis-issued certificate (e.g. a CA issues a cert for bbc.co.uk without authorisation) becomes detectable within minutes of being used — even before any user is harmed.
	Search by exact domain:
	`bbc.co.uk`
	Wildcard search:
	`%.bbc.co.uk`
	Results show: issuer CA, SAN list, validity dates, log entry timestamps.
		Add a CAA DNS record to restrict which CAs may issue for your domain:
		`bbc.co.uk. CAA 0 issue "digicert.com"`
		Subscribe to CT monitoring (e.g. via sslmate.com/certspotter) to receive alerts when any new certificate is issued for your domain.

A08 Software & Data Integrity Failures — Four Attack Patterns

Module 9 — A04 & A08 Quick Reference Card

BBC Academy / JBI Training

Pattern	What the Attacker Does	Root Cause	Defence
BASKET / PRICE TAMPERING	Intercepts PUT /api/BasketItems and modifies price, quantity, or BasketId field.	Server trusts client-supplied values without verifying them against server-side state.	Never calculate price or totals on the client. Re-derive all values server-side from the product catalogue. Verify that BasketId matches session user before any write.
JWT TOKEN FORGERY	Decodes JWT, changes role/email claim, re-signs with known/weak secret, or strips signature (alg:none).	Weak or hardcoded HMAC secret; library configured to accept algorithm downgrade.	32+ byte cryptographically random secret from environment. Explicit algorithm allowlist in decode(). Short expiry + refresh tokens. Rotate secret on any suspected exposure.
COUPON / HASH MANIPULATION	Reverse-engineers MD5-based coupon scheme, generates arbitrary valid coupons (e.g. forge 90% discount).	Coupon 'signature' is a predictable MD5 hash — no secret key, no expiry, no server-side revocation list.	HMAC-SHA256(coupon_data, server_secret). Validate every coupon server-side against a signed token with expiry and single-use flag. Store used codes in DB.
MALICIOUS FILE UPLOAD	Uploads a PHP/Python shell disguised as image.jpg. Bypasses client-side MIME check by modifying Content-Type header in transit.	Server trusts client-supplied Content-Type. Does not inspect file magic bytes. Uploaded files served from same origin as app.	Check file magic bytes server-side (python-magic). Rename uploaded files to UUID. Serve uploads from separate origin with Content-Disposition: attachment. Apply strict CSP. Never execute uploaded files.

Seven Questions to Detect A04 & A08 Failures at Design Time

#	Design-Time Question	If You Cannot Answer → Likely Failure Class
1	**Is any sensitive data (passwords, session tokens, PII) transmitted without TLS 1.2+?**	A04 — transport failure; HSTS not enforced
2	**Is any password stored as MD5, SHA-1, or unsalted SHA-256?**	A04 — weak hashing; bcrypt / Argon2 required
3	**Is any secret or encryption key hard-coded in source code or a committed config file?**	A04 — key exposure; use environment variables, secret managers
4	**Does the server re-derive prices and totals from its own catalogue before completing any transaction?**	A08 — client-trusted value; all financial calculations must be server-side
5	**Are JWT or session tokens signed with a cryptographically random secret of at least 32 bytes?**	A08 — token forgery; alg:none or weak HMAC secret
6	**Does the server check file magic bytes (not just extension/MIME type) before accepting uploads?**	A08 — malicious upload; client MIME is attacker-controlled
7	**Are software updates (packages, Docker images, CI artefacts) verified with a cryptographic signature or hash before execution?**	A08 — supply chain integrity; use signed releases, SLSA framework

A04 — The Two Principles

Principle 1: Never implement your own cryptography

Every real-world break in the last decade came from a developer who thought their own XOR, ROT13, or 'custom cipher' was 'good enough', or who combined primitives incorrectly.

→ **Find a well-reviewed library that handles your use case. Use its defaults.**

Principle 2: Never make your own cryptographic decisions

ECB mode is available in every library — it has legitimate uses for single-block encryption of random data. The library cannot know your data is structured and multi-block.

A08 — The Two Rules

Rule 1: Never trust client-supplied values for security decisions

Prices, basket totals, user IDs, file types, and role claims that arrive from the browser are all attacker-controlled. Treat them as untrusted input — re-derive or re-verify every one server-side.

→ **The server owns the truth. The client displays it.**

Rule 2: Sign or hash every piece of data whose integrity matters

→ Use the library's recommended default, not the most convenient option. Fernet uses AES-128-CBC. You do not need to choose.	If an attacker can modify a value in transit or storage and the server will act on it, that value needs an integrity mechanism: HMAC, digital signature, or server-side re-derivation. → Ask: 'If an attacker changes this value, what can they gain?' If the answer is 'anything useful', add an integrity check.

Module 9 Tool Reference — What You Used & Where to Go Next

Tool / Resource	Used For in Module 9	Key Finding / Result	Where to Explore Further
ZAP Proxy	HTTPS interception (MITM)	Full plaintext of every HTTPS request visible once rogue CA installed	ZAP: Tools → Active Scan on Juice Shop to find additional A04/A08 issues
crt.sh	Certificate Transparency search	All certs issued for a domain — detects mis-issuance	Search %.yourdomain.com monthly; add CT monitoring alert
tldr.fail	TLS version deprecation reference	RFC 8996 timeline; browsers/servers still supporting TLS 1.0/1.1	Check your own org's servers at: ssllabs.com/ssltest
CyberChef	Decode/analyse coupon encoding	Base85(Z85) + MD5 revealed weak coupon signing	gchq.github.io/CyberChef — Recipe: 'Magic' auto-detects encoding
jwt.io	JWT decode & forge	alg claim, role/email payload visible; weak secret identified	portswigger.net/web-security/jwt — interactive JWT labs
DVWA	Malicious file upload	PHP shell executed via upload path — remote code execution achieved	DVWA Medium/High levels show progressive hardening techniques
Juice Shop	All A04 & A08 challenges	Coupon bypass, basket tampering, JWT forgery, admin panel access	pwning.owasp-juice.shop — full challenge walkthrough guide

A04 — Notable Real-World Incidents

Adobe (2013)

153 million passwords stored as 3DES with the same key and no salt. Password hints stored in plaintext alongside them — effectively making the hashes crackable by crowd-sourcing the hints.

LinkedIn (2012)

6.5 million SHA-1 unsalted password hashes posted publicly. 90% cracked within days. SHA-1 at the time was already considered weak for password storage.

Facebook (2019)

Hundreds of millions of passwords stored in plaintext in internal logs, accessible to ~20,000 engineers. No encryption or hashing applied.

A08 — Notable Real-World Incidents

SolarWinds (2020)

Attackers inserted malicious code into the Orion software build pipeline. Signed updates were distributed to ~18,000 organisations. Integrity of the build artefact was never independently verified.

event-stream / npm (2018)

A malicious package was injected into the npm supply chain via a maintainer hand-off. The compromised package was downloaded ~8 million times and targeted cryptocurrency wallets.

Equifax (2017)

A known Apache Struts vulnerability (CVE-2017-5638) was not patched for 2 months after disclosure. The failure to verify integrity of deployed components led to breach of 147 million records.

Keep this card after the course. For any new feature or endpoint: run the seven design-time questions above before writing a line of code. If you cannot answer Q1–Q3, review A04. If you cannot answer Q4–Q7, review A08.